Divide and Conquer in Decentralized AI Training Coordination

Master's thesis · Two students · Six months

Background

Training large AI models needs substantial computing power. Platforms such as Hivemind let independent peers share resources. Hivemind uses a distributed hash table (DHT) to find peers and libp2p for communication. Peers form temporary groups to aggregate model updates.

Byzantine-robust aggregation limits the influence of malicious model updates under assumptions about the workers contributing to an aggregation. Krum requires $n > 2f + 2$, where n is the total number of contributing workers and f is an upper bound on the number of malicious workers. For example, with seven workers, this condition allows at most two malicious workers. Its guarantee also depends on statistical assumptions about honest gradients, including their variability. In temporary groups, discovery and admission affect who joins. Timeouts and dropouts affect who actually contributes.

Literature gaps

Prior work studies malicious model updates and attacks on peer selection. For the platforms in this thesis, two questions remain about how discovery, admission, timeouts and recovery work together:

- Which forged or delayed messages change group membership or actual contributors, and how do these changes affect aggregation and training?
- Can protections from IPFS and Ethereum's networking stack, as implemented in go-ethereum, prevent these attacks? What effects do they have on latency, resource use and honest-peer participation?

Goals

- Determine how discovery, group formation, admission, timeouts and recovery affect participation across three open-source DHT/libp2p training platforms. Select one for detailed study.
- Determine how the global malicious share relates to the share among a group's actual contributors, and whether forged or delayed messages disrupt rounds or change training results.
- Assess whether protections from IPFS and go-ethereum apply to training coordination, and determine the effect and cost of one mitigation in the primary platform.

Milestones

Months 1–2: Scope and baseline

- Choose three platforms and the primary case with the supervisor. Review relevant literature.
- Map coordination code, define the attacker model, and build a private testbed with an honest baseline.

Months 2–4: Controlled experiments

- Test forged and delayed messages across attacker shares, message budgets, group sizes and delays. Record peers, contributors, round outcomes, learning and existing checks.

Months 4–5: Mitigation and evaluation

- Assess IPFS and go-ethereum mechanisms. Implement one and compare matched runs with it off and on. Measure attack effects, delay, traffic, resource use and effects on honest peers.

Month 6: Analysis and writing

- Analyse the results, document the experiments and write the thesis.

Expected research outcomes

- Test results showing which messages change group membership, contribution weights, aggregation completion or training performance, and which checks block them.
- Measurements of the attacker fraction within groups at different global attacker fractions and message budgets.
- Measurements of how the mitigation changes attack success, latency, CPU, memory and network traffic.

Research deliverables

A source-code comparison of three platforms, a private testbed with experiment settings, traces and analysis, one mitigation implementation and evaluation, and the thesis.

Learning outcomes

Students will learn to:

- Trace peer discovery and aggregation in code and distinguish group membership from actual contributions.
- Build a private multi-peer testbed and run controlled, repeatable experiments.
- Analyse participation, availability, learning and resource data, and state what the results support.
- Implement a mitigation and measure its effects and costs against a baseline.

Desired qualifications

- Strong programming skills, preferably Python, and basic networking and distributed-systems knowledge.
- Linux and container experience is useful. Network security or distributed systems coursework is beneficial. Basic machine-learning knowledge is helpful.